

Programmation Objet Avancée

Interfaces graphiques avec JavaFX

Mohamed ZAYANI

2025/2026

Plan

- **Présentation de JavaFX**
- **Caractéristiques d'une application JavaFX**
- **Notion Stage/Scene**
- **Composants graphiques JavaFX**
- **Notion de Layout (Gestionnaire de mise en page)**
- **Structure d'une application JavaFX**
- **Gestion des événements**
- **Interfaces graphiques JavaFX avec FXML**

Présentation de JavaFX

- **JavaFX** est une plate-forme logicielle pour créer et fournir des applications de bureau, ainsi que des applications Internet riches.
- Les applications intégrées à **JavaFX** peuvent s'exécuter sur plusieurs plateformes, notamment Web, Mobile et Desktops.
- **JavaFX** est la dernière génération de bibliothèques qui permet de créer des interfaces graphiques de qualité pour Java.
- Avec **Java 1.8**, **JavaFX** est devenue la bibliothèque officielle de Java pour la création des interfaces graphiques.
- Le développement de son prédécesseur **Swing** est abandonné (sauf pour les corrections des bogues)

Caractéristiques d'une application JavaFX (1)

- Une application **JavaFX** est une classe dérivée de la classe abstraite **Application** du package **javafx.application**.

```
import javafx.application.Application;

public class App1JavaFX extends Application{

}
```

- Il est essentiel, ainsi, d'implémenter une méthode **start** (de la classe **Application**) qui présente le point d'entrée principal du fonctionnement de toute application **JavaFX**.
- La méthode **start** contient un argument de type **Stage** qui présente la fenêtre principale de l'application:

```
import javafx.application.Application;
import javafx.stage.Stage;

public class App1JavaFX extends Application{
    @Override
    public void start (Stage primaryStage) throws Exception {
    }
}
```

Caractéristiques d'une application JavaFX (2)

- Pour lancer l'exécution de l'application **JavaFX**, on appelle une autre méthode de la classe **Application** nommée **launch** (en lui passant les arguments de la méthode **main**) pour démarrer la création graphique:

```
import javafx.application.Application;
import javafx.stage.Stage;

public class App1JavaFX extends Application {

    public static void main(String[] args) {
        Launch(args);
    }

    @Override
    public void start(Stage primaryStage) throws Exception {
    }
}
```

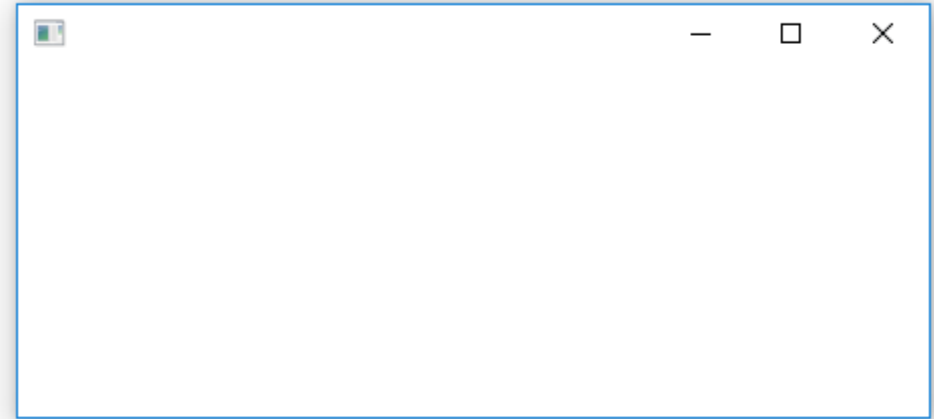
- L'exécution de la classe **App1JavaFX** démarre l'application graphique et la fenêtre est créée...

mais rien n'est affiché...

Caractéristiques d'une application JavaFX (3)

- Pour afficher la fenêtre graphique, il est nécessaire d'appeler la méthode **show** de l'objet de type **Stage** comme ceci: Le résultat de l'exécution est le suivant:

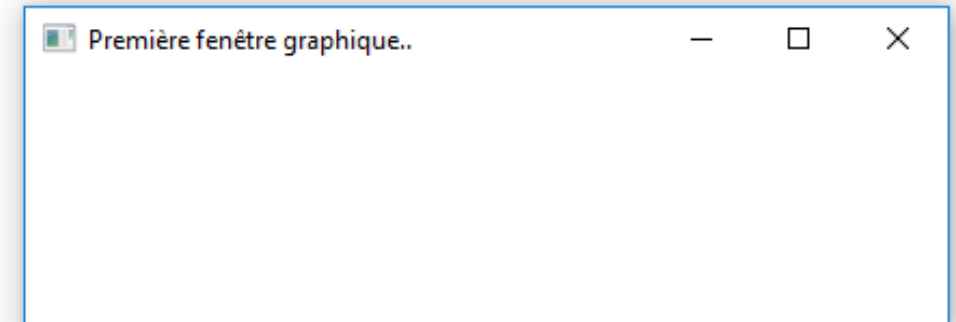
```
@Override
public void start(Stage primaryStage) throws Exception {
    primaryStage.show();
}
```



- Il est possible de personnaliser l'affichage en utilisant les méthodes de l'objet de type Stage.
- **Exemple: Spécifier un titre à la fenêtre**

```
@Override
public void start(Stage primaryStage) throws Exception {
    primaryStage.setTitle("Première fenêtre graphique..");
    primaryStage.show();
}
```

Le résultat de l'exécution est le suivant:



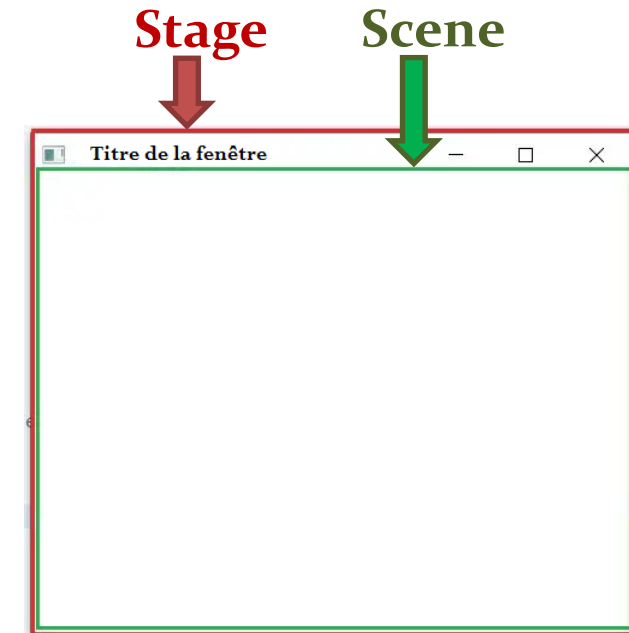
Éléments de base d'une application JavaFX

- Une application **JavaFX** s'appuie essentiellement sur **deux** principaux éléments (**Stage** et **Scene**) qui constituent le support de tous les composants graphiques:

1. Un composant **Stage** qui représente la fenêtre principale de l'application et présente des méthodes pour configurer :

- La dimension de la fenêtre
- Sa position par rapport à l'écran
- Son affichage
- Son titre
- Sa fermeture
- Son contenu (à travers un objet Scene)
- Etc,

2. Un composant **Scene** qui permet d'afficher tout ce qui devrait apparaître dans l'application: l'objet **Scene** contient, dans un objet **SceneGraph**, tous les composants graphiques organisés d'une manière hiérarchique.



Un stage peut être vu comme un théâtre (espace physique) dans lequel, il est possible de regarder, à un instant donné, une scène composée d'un ensemble d'acteurs (les composants graphiques)

Composants graphiques d'une application JavaFX

- Les composants graphiques sont des objets qui peuvent être de différents types:
 - Des éléments de contrôle utilisateur: **Label**, **TextField**, **ListView** ...
 - Des formes graphiques: **Circle**, **Rectangle**, **Line** ...
 - Des médias: **ImageView**, **MediaView** ...
 - Des graphiques: **PieChart**, **LineChart**, **BarChart** ...
 - Des layouts pour grouper les éléments pour assurer la mise en page : **BorderPane**, **Hbox**, **Vbox**, **GridPane** ...

Description des principaux widgets (composants graphiques)

- **Label** : Utilisé pour afficher des étiquettes de texte simples.
- **Button**: Utilisé pour déclencher un événement qui provoquer un traitement donnée
- **TextField** : Utilisé pour prendre l'entrée de l'utilisateur sur une seule ligne.
- **RadioButton**: Présente à l'utilisateur certaines options représentées par RadioButtons. Une seule option peut être sélectionnée.
- **CheckBox**: Présente à l'utilisateur certaines options représentées par des cases à cocher. Plusieurs options peuvent être sélectionnées.
- **ComboBox**: Présente à l'utilisateur une liste déroulante à partir de laquelle une option peut être sélectionnée.

Description des principaux widgets (composants graphiques)

- **TextArea**: Utilisé pour saisir des textes multilignes de l'utilisateur.
- **MenuButton**: Présente à l'utilisateur une liste d'options sous la forme d'un bouton et d'une liste déroulante.
- **ImageView**: Utilisé pour afficher des images dans la fenêtre de l'interface graphique JavaFX.
- **PasswordField** : Un champ de saisie spécial pour prendre les mots de passe en entrée.
- **FileChooser** : Utilisé pour sélectionner un fichier à partir de l'ordinateur local.
- **Input Dialogs** : Présente une boîte de dialogue à l'utilisateur, renvoyant l'entrée de l'utilisateur.
- **Alert Dialogs** : Présente à l'utilisateur une « alerte » sous la forme d'une boîte de dialogue.

Gestionnaire de mise en page (Layout)

- Un **layout** ou gestionnaire de mise en page est un noeud graphique qui hérite de la classe `javafx.scene.layout.Pane`.
- Il s'agit d'un nœud dans l'hiérarchie qui contient d'autres noeuds et qui est chargée de:
 - les déplacer,
 - de les disposer,
 - voire de les redimensionner de manière à changer la présentation de cet ensemble de nœuds
 - et à les rendre utilisables dans une interface graphique.
- Un **layout** est une classe de l'API Graphique permettant d'organiser les objets graphiques dans la fenêtre
- Un **layout** est un noeud graphique et peut donc être positionné et manipulé, ou subir des transformations et effets comme n'importe quel autre composant graphique

Types de layout

- En JavaFX, il existe **huit** types de layouts :
1. Le **BorderPane** qui permet de diviser une zone graphique en **cinq** parties : top, down, right, left et center.
 2. La **HBox** qui permet d'aligner horizontalement les éléments graphiques.
 3. La **VBox** qui permet d'aligner verticalement les éléments graphiques.
 4. Le **StackPane** qui permet de ranger les éléments de façon à ce que chaque nouvel élément inséré apparaisse au-dessus de tous les autres.
 5. Le **GridPane** permet de créer une grille d'éléments organisés en lignes et en colonnes
 6. Le **FlowPane** permet de ranger des éléments de façon à ce qu'ils se positionnent automatiquement en fonction de leur taille et de celle du layout.
 7. Le **TitlePane** est similaire au FlowPane, chacune de ses cellules fait la même taille.
 8. L'**AnchorPane** permet de fixer un élément graphique par rapport à un des bords de la fenêtre : top, bottom, right et left.

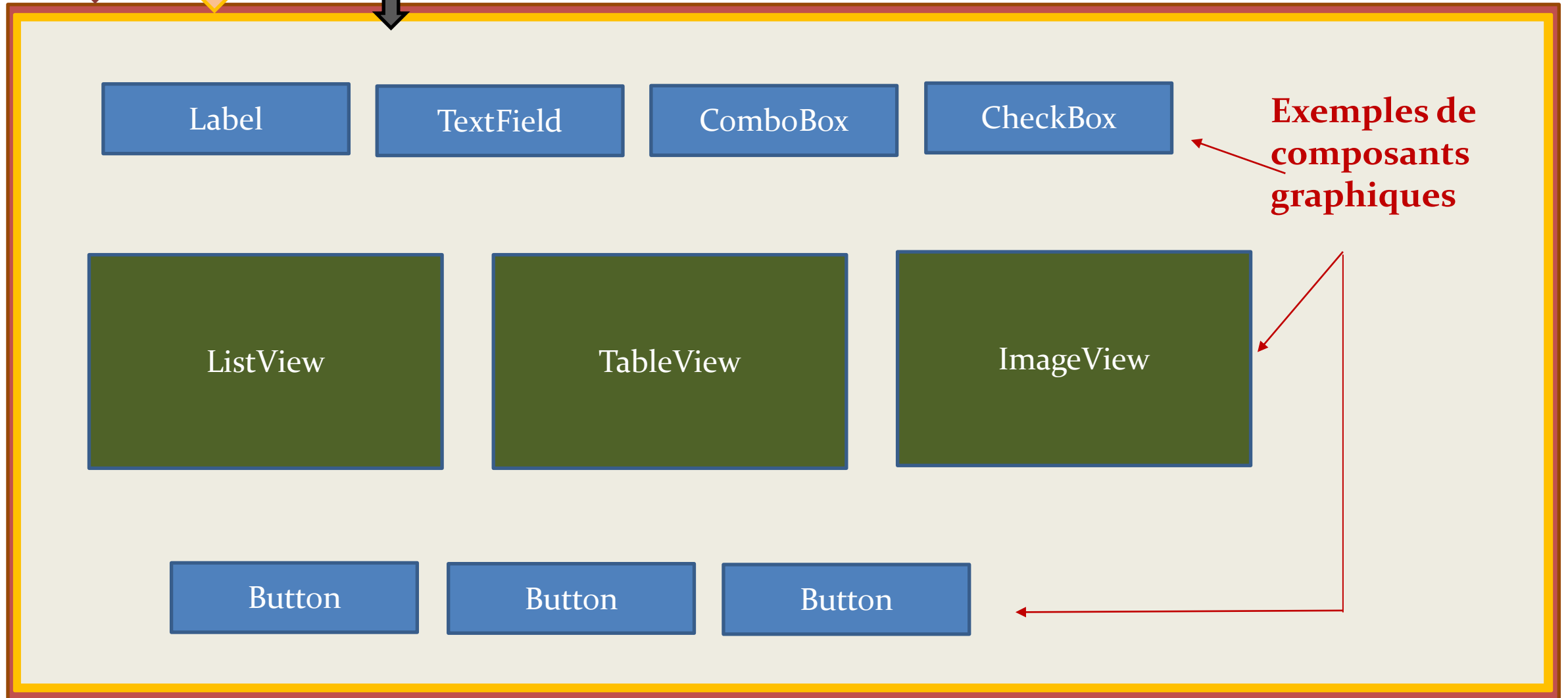
Structure d'une application JavaFX

Stage

Scene

Layout : conteneur pour gérer la disposition des composants

Il est possible d'utiliser des layouts imbriqués pour améliorer la mise en page



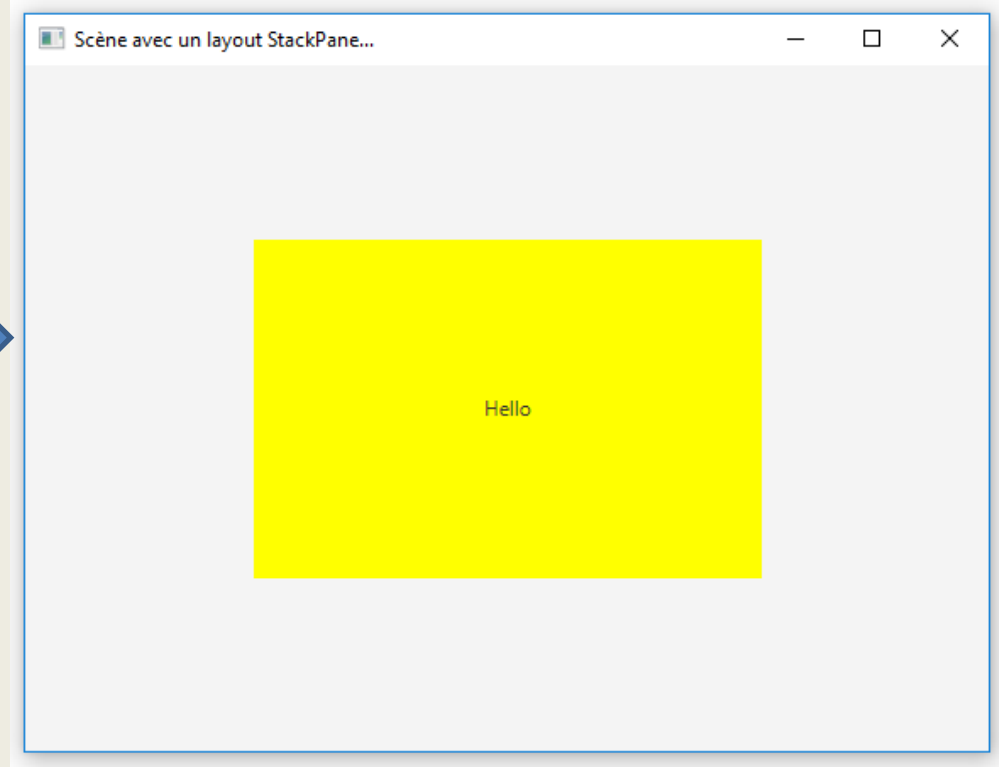
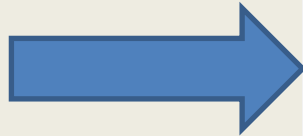
Exemple de composants en mode StackPane

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.layout.StackPane;
import javafx.scene.control.Label;
import javafx.scene.shape.Rectangle;
import javafx.scene.paint.Color;
import javafx.scene.Scene;

public class App2JavaFX extends Application{

    public static void main(String[] args) {
        Launch(args);
    }

    @Override
    public void start(Stage primaryStage) throws Exception {
        //layout parent pour une disposition en pile chaque nouvel
        // élément inséré apparaît au-dessus
        StackPane root = new StackPane();
        //Etiquette
        Label labelMessage = new Label("Hello");
        //Rectangle vert (300,200)
        Rectangle rectangleJaune = new Rectangle(300,200,Color.YELLOW);
        //Ajouter le premier noeud fils au layout parent
        root.getChildren().add(rectangleJaune);
        //Ajouter le deuxième noeud fils au layout parent
        root.getChildren().add(labelMessage);
        //Créer une scène avec une dimension 600, 400 et lui affecter le layout parent
        Scene scene = new Scene (root, 600,400);
        //Associer la scene à l'objet Stage "primaryStage"
        primaryStage.setScene(scene);
        //Spécifier le titre de la fenêtre
        primaryStage.setTitle("Scène avec un layout StackPane...");
        //Afficher la fenêtre
        primaryStage.show();
    } }
```



- Le Label est affiché au dessus du Rectangle
- Si on inverse l'ordre d'ajout des composants , le Label sera au dessous et ne sera pas visible.
- Dans ce cas, il est possible d'utiliser la méthode `ToFront()` du Label pour qu'il soit au dessus des autres

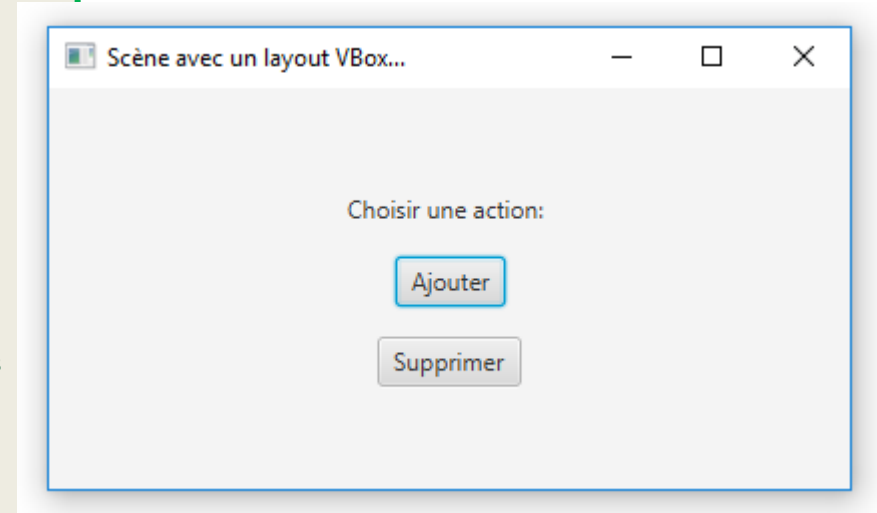
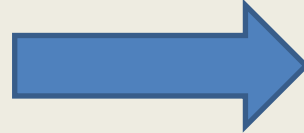
Exemple de composants en mode VBox

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.layout.VBox;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.control.Label;
import javafx.scene.control.Button;
import javafx.scene.Scene;

public class App3JavaFX extends Application{

    public static void main(String[] args) {
        Launch(args);
    }

    @Override
    public void start(Stage primaryStage) throws Exception {
        //layout parent pour une disposition Verticale: chaque nouvel élément inséré apparaît en bas de tous les autres
        VBox root = new VBox();
        // Spécifier une marge de 10 entre les composants et le panneau
        root.setPadding(new Insets(10));
        // Spécifier un espacement de 15 entre les composants
        root.setSpacing(15);
        //centrer les composants
        root.setAlignment(Pos.CENTER);
        //Etiquette
        Label labelMessage = new Label(); labelMessage.setText("Choisir une action: ");
        //Bouton "Ajouter"
        Button boutonAjouter =new Button("Ajouter");
        //Bouton "Supprimer"
        Button boutonSupprimer =new Button("Supprimer");
        //ajouter tous les composants graphiques à leur parent
        root.getChildren().addAll(labelMessage,boutonAjouter,boutonSupprimer);
        //Créer une scène avec une dimension 400,200 et lui affecter le layout parent
        Scene scene = new Scene (root,400,200);
        //Associer la scene à l'objet Stage "primaryStage"
        primaryStage.setScene(scene);
        //Spécifier le titre de la fenêtre
        primaryStage.setTitle("Scène avec un layout VBox...");
        //Afficher la fenêtre
        primaryStage.show();
    } }
```

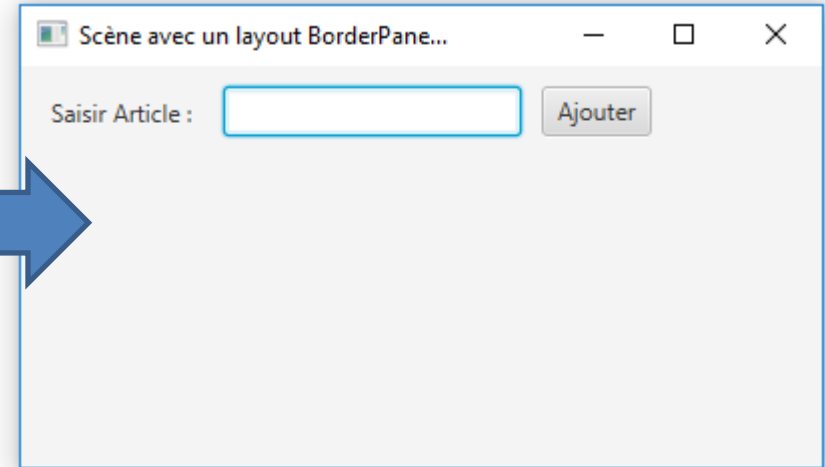


- L'ordre des composants enfants dépend de leurs ajouts au parent

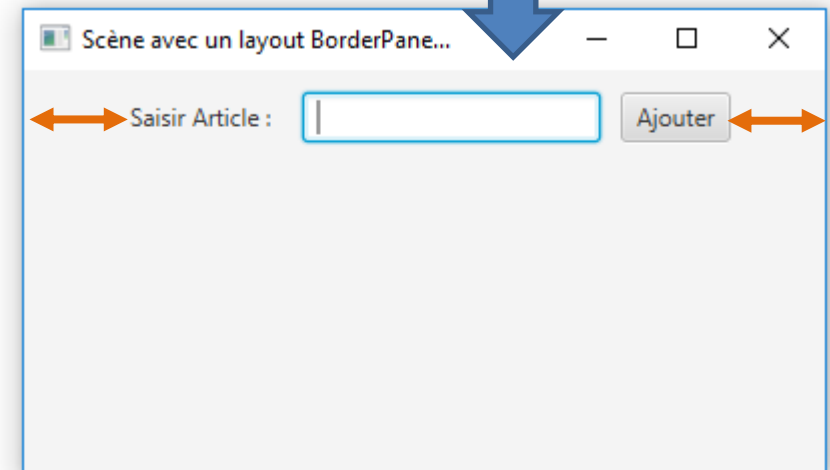
Exemple de composants en mode BorderLayout(1)

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.layout.BorderPane;
import javafx.geometry.Insets;
import javafx.scene.layout.HBox;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.control.Button;
import javafx.scene.Scene;

public class App4JavaFX extends Application{
    public static void main(String[] args) {        Launch(args); }
    @Override
    public void start(Stage primaryStage) throws Exception {
        //layout parent pour une disposition BorderLayout
        //chaque nouvel élément inséré occupe une des cinq zones prédéfinies (Top, Left, Center, Right, Bottom)
        BorderPane root = new BorderPane();
        // Spécifier une marge de 10 entre les composants et le panneau
        root.setPadding(new Insets(10));
        //layout enfant pour une disposition horizontale avec un espacement de 10 entre ces composants
        HBox layoutTop = new HBox(10);
        //Etiquette
        Label labelArticle = new Label("Saisir Article :");
        // Spécifier une marge de 5 entre le texte et le cadre du Label
        labelArticle.setPadding(new Insets(5));
        //Zone de texte
        TextField texteArticle = new TextField();
        //Bouton "Ajouter"
        Button boutonArticle = new Button("Ajouter");
        //affecter les composants de la zone "Top" à leur layout
        layoutTop.getChildren().addAll(labelArticle, texteArticle, boutonArticle);
        // affecter le layout à la zone "Top" du parent
        root.setTop(layoutTop);
        //Créer une scène avec une dimension et lui affecter le layout parent
        Scene scene = new Scene (root,400,200);
        //Associer la scene à l'objet Stage "primaryStage"
        primaryStage.setScene(scene);
        //Spécifier le titre de la fenêtre
        primaryStage.setTitle("Scène avec un layout BorderLayout...");
        //Afficher la fenêtre
        primaryStage.show();
    } }
```



- Il est possible de centrer les composants de la zone « Top » en utilisant l'instruction: `layoutTop.setAlignment(Pos.CENTER);`



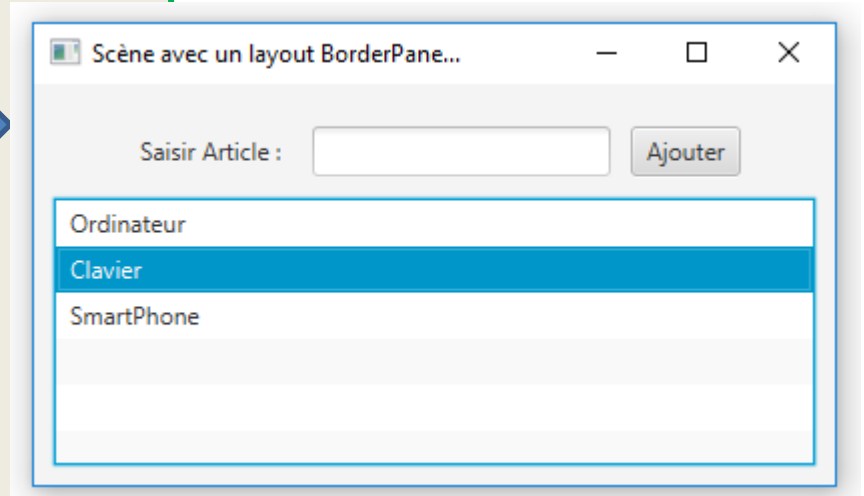
Exemple de composants en mode BorderLayout(2)

Ajouter une liste dans la zone « Center »

```
// affecter le layout à la zone "Top" du parent
root.setTop(layoutTop);

// Gérer la zone "Center"
//layout enfant pour une disposition horizontale
VBox layoutCenter = new VBox();
// Définir une marge de 10 entre les bordures du layout et ses composants
layoutTop.setPadding(new Insets(10));
//Liste
ListView<String> listeArticles = new ListView<String>();
//Ajouter des éléments à la liste
listeArticles.getItems().addAll("Ordinateur", "Clavier", "SmartPhone");
//Définir la liste comme un enfant pour layoutCenter
layoutCenter.getChildren().add(listeArticles);
//Associer le layout à la zone "Center" du root
root.setCenter(layoutCenter);

//Créer une scène avec une dimension et lui affecter le layout parent
Scene scene = new Scene (root,400,200);
//Associer la scene à l'objet Stage "primaryStage"
primaryStage.setScene(scene);
//Spécifier le titre de la fenêtre
primaryStage.setTitle("Scène avec un layout BorderLayout...");
//Afficher la fenêtre
primaryStage.show();
} }
```

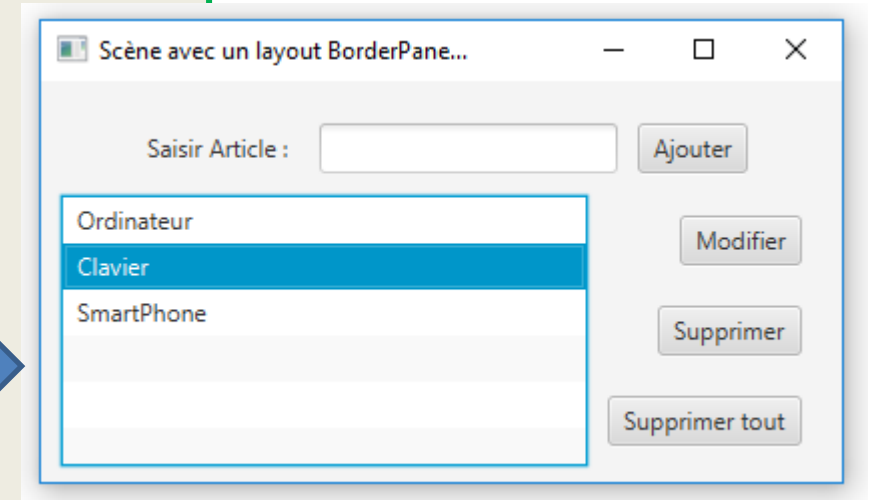


Exemple de composants en mode BorderLayout(3)

```
//Associer le layout à la zone "Center" du root
root.setCenter(layoutCenter);

//----- Gérer la zone "Right"-----
//layout enfant pour une disposition verticale
VBox layoutRight = new VBox();
// Définir un espacement de 20 entre les composants
layoutRight.setSpacing(20);
//Aligner les composants du layout à droite
layoutRight.setAlignment(Pos.CENTER_RIGHT);
// Définir une marge de 10 entre les bordures du layout et ses composants
layoutRight.setPadding(new Insets(10));
//Bouton "Modifier"
Button boutonModifier = new Button("Modifier");
//Bouton "Supprimer"
Button boutonSupprimer = new Button("Supprimer");
//Bouton "SupprimerTout"
Button boutonSupprimerTout = new Button("Supprimer tout");
//Ajouter les composants au layout
layoutRight.getChildren().add(boutonModifier);
layoutRight.getChildren().add(boutonSupprimer);
layoutRight.getChildren().add(boutonSupprimerTout);
//Associer le layout à la zone "Right" du root
root.setRight(layoutRight);
//Créer une scène avec une dimension et lui affecter le layout parent
Scene scene = new Scene (root,400,200);
```

Ajouter des boutons dans la zone « Right »



Gestion des événements

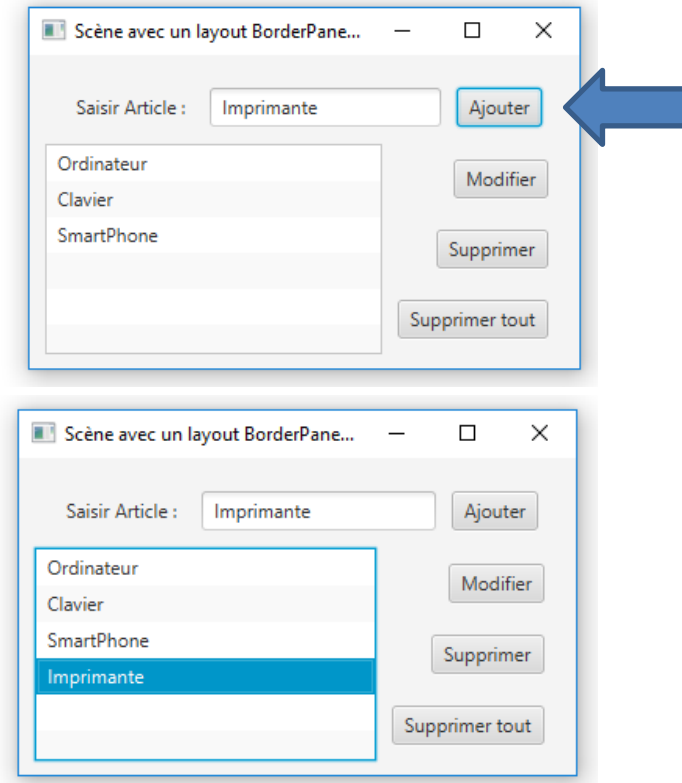
- Par défaut, les composants graphiques sont **sourds**: ils ne réagissent pas aux actions (événements) des utilisateurs.
- Il est essentiel d'ajouter à un type d'événement (**appui sur bouton**, **click de souris**, **saisie d'un caractère...**) un écouteur (listener) ou bien un observateur (autrement dit **enregistrer un gestionnaire d'événement**)
- L'observateur sera notifié de l'arrivée d'un événement (**event**) et par la suite une méthode spécifique de l'observateur sera déclenchée pour gérer l'événement et réaliser le traitement adéquat.
- **Par exemple**: pour programmer le comportement d'un **Button** suite à un action de l'utilisateur, il suffit de lui associer un objet (**écouteur**) qui implémente une interface de type **EventHandler** et redéfinir sa méthode **handle**

Exemple d'un bouton cliqué (1)

- Reprendre l'exemple précédent pour programmer l'ajout d'un article saisi par l'utilisateur dans la liste suite à l'appui sur le bouton « **Ajouter** »:
- Première solution:** définir l'écouteur comme une classe interne (à l'intérieur du code de la classe de l'application graphique) qui implémente l'interface « **EventHandler** » et qui redéfinit la méthode **handle** (cette méthode reçoit l'objet **ActionEvent** déclenché suite à l'appui sur le bouton « **Ajouter** » et réalise le traitement d'ajout de l'article saisi dans la liste)

```
//Une classe interne
private class AjoutArticleEcouteur implements EventHandler<ActionEvent>
{
    @Override
    public void handle(ActionEvent e) {
        //récupérer le texte saisi par l'utilisateur dans le composant TextField
        // Les composants graphiques doivent être des attributs graphiques pour qu'ils soient
        // visibles par la classe interne
        String article= texteArticle.getText();
        //Insérer l'article dans la liste des éléments de la ListView
        listeArticles.getItems().add(article);
    }
}
```

```
//dans le code de la classe graphique
primaryStage.show();
// Associer un écouteur pour l'action sur le bouton « Ajouter »
boutonArticle.setOnAction (new AjoutArticleEcouteur() );
```

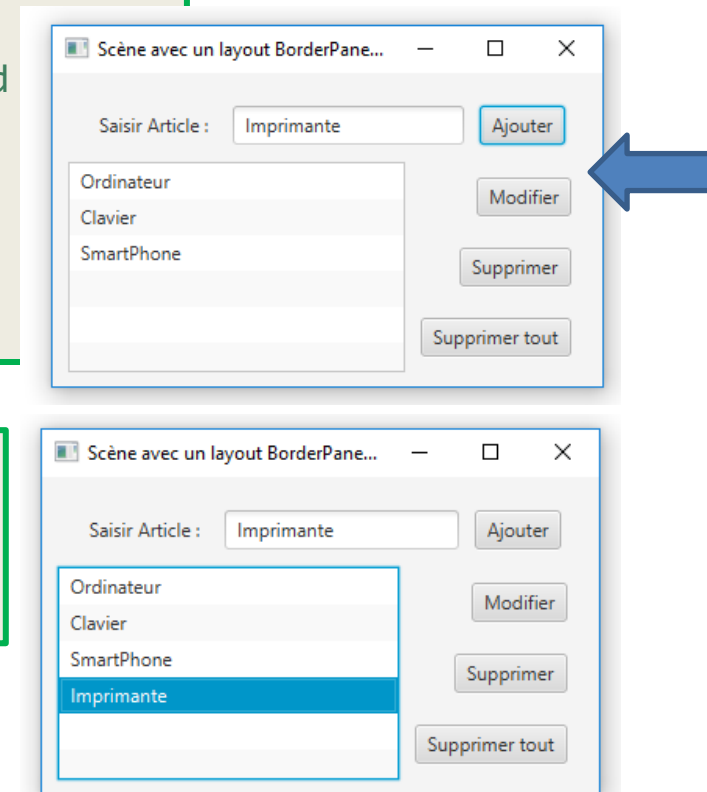


Exemple d'un bouton cliqué (2)

- **Deuxième solution:** définir la classe graphique elle même comme écouteur càd implémente l'interface « **EventHandler** » et qui redéfinit la méthode **handle** (cette méthode reçoit l'objet **ActionEvent** déclenché suite à l'appui sur le bouton « **Ajouter** » et réalise le traitement d'ajout de l'article saisi dans la liste)

```
// classe graphique et écouteur en même temps
Public class App8JavaFX extends Application implements EventHandler<ActionEvent>
{
    @Override
    public void handle(ActionEvent e) {
        //récupérer le texte saisi par l'utilisateur dans le composant TextField
        String article= texteArticle.getText();
        //Insérer l'article dans la liste des éléments de la ListView
        listeArticles.getItems().add(article);
    }
}
```

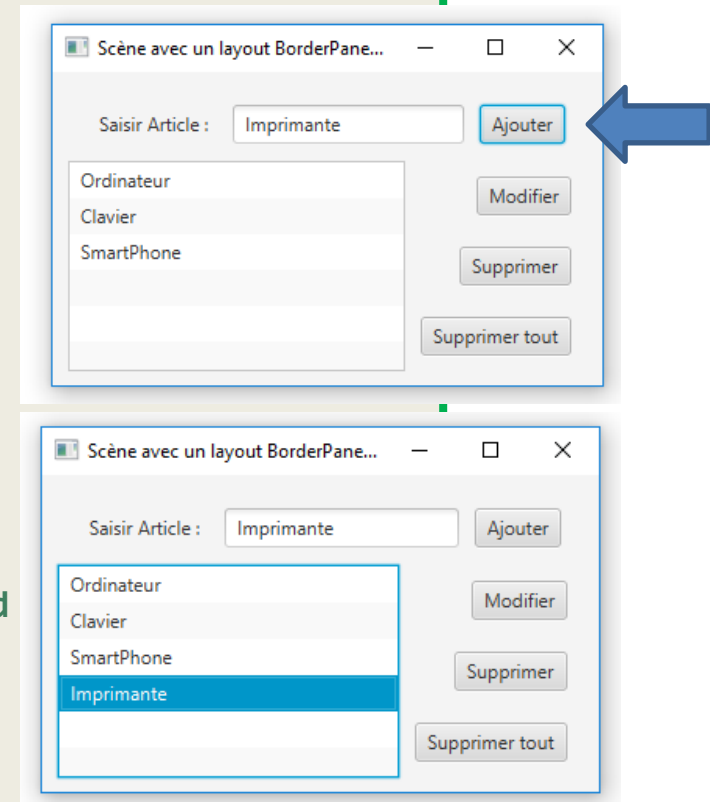
```
//dans le code de la classe graphique
primaryStage.show();
// Associer l'objet courant comme écouteur pour l'action sur le bouton « Ajouter »
boutonArticle.setOnAction (this );
```



Exemple d'un bouton cliqué (3)

- **Troisième solution:** utiliser une classe anonyme comme écouteur càd implémente l'interface « **EventHandler** » et qui redéfinit la méthode **handle**

```
//Afficher la fenêtre
primaryStage.show();
// gérer l'ajout d'un article saisi et son insertion dans la liste
boutonArticle.setOnAction
(
    new EventHandler<ActionEvent>() // classe anonyme
    {
        @Override
        public void handle(ActionEvent e)
        {
            //récupérer le texte saisi par l'utilisateur dans le composant TextField
            String article= texteArticle.getText();
            //Insérer l'article dans la liste des éléments de la ListView
            listeArticles.getItems().add(article);
        }
    }
);
```



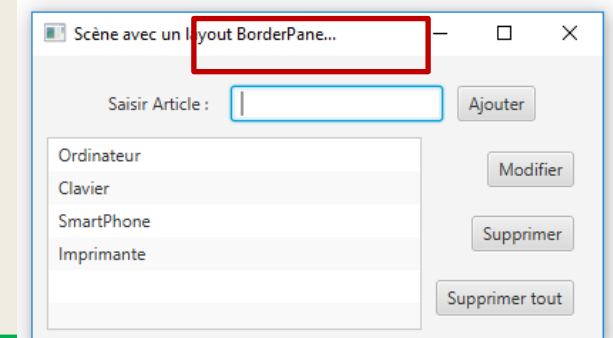
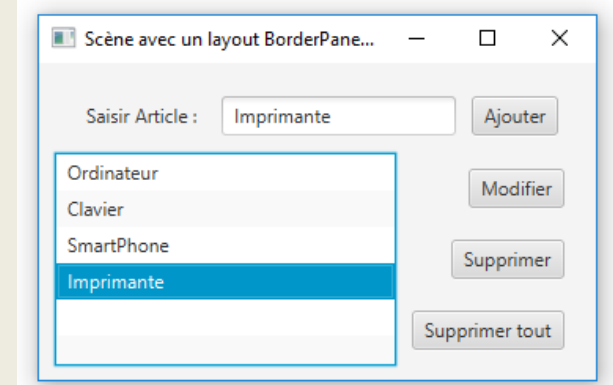
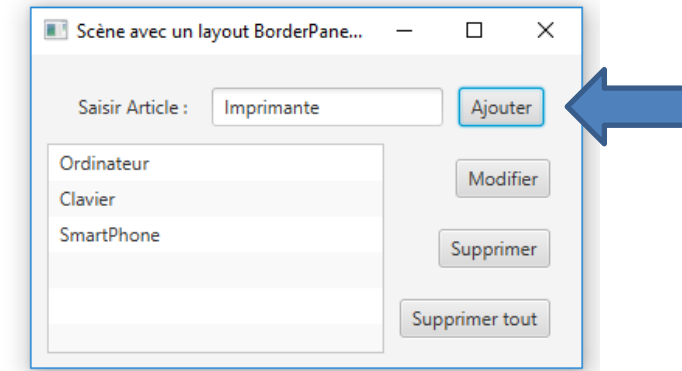
- Pour vider la zone de saisie après avoir appuyé sur le bouton Ajouter, il suffit d'écrire l'instruction suivante à la fin de la méthode handle:

texteArticle.clear();

Exemple d'un bouton cliqué (4)

- **Quatrième solution:** utiliser les expressions **lamda** pour définir un écouteur qui implémente l'interface « **EventHandler** » et qui redéfinit la méthode **handle**

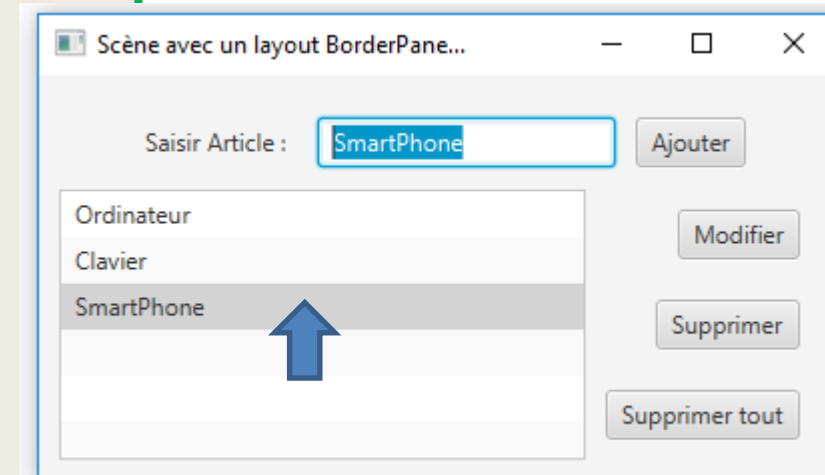
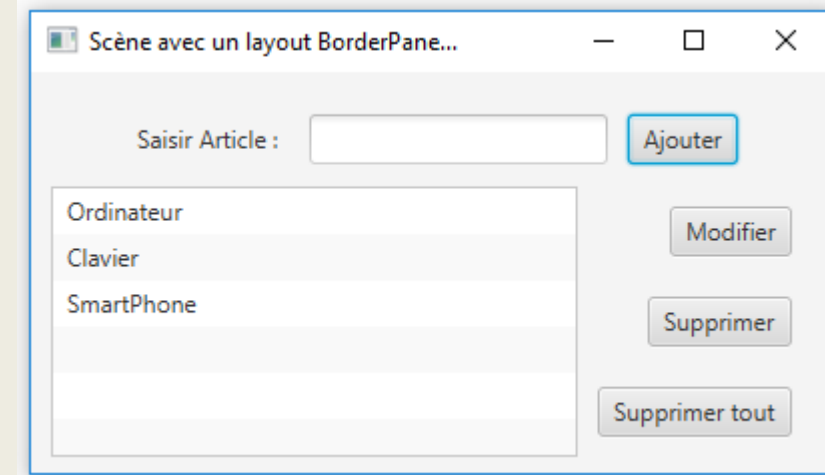
```
//Afficher la fenêtre
primaryStage.show();
// gérer l'ajout d'un article saisi et son insertion dans la liste
boutonArticle.setOnAction
(
    (e)-> //expression lamda
    {
        //récupérer le texte saisi par l'utilisateur dans le composant TextField
        String article= texteArticle.getText();
        //Insérer l'article dans la liste des éléments de la ListView
        listeArticles.getItems().add(article);
        // vider la zone de texte
        texteArticle.clear();
        // donner le focus à la zone de saisie
        texteArticle.requestFocus();
    }
);
```



Exemple d'un double click sur ListView

- Le double click sur un élément de la liste permet de copier la valeur de cet élément dans la zone de saisie:

```
// Gérer la sélection d'un élément dans ListView
listeArticles.setOnMouseClicked( (e)->
{
    // s'il s'agit du bouton gauche de la souris et un double click
    if ( (e.getButton()==MouseButton.PRIMARY)
        && (e.getClickCount()==2))
    {
        //récupérer l'élément sélectionné
        String articleSelectionne =
listeArticles.getSelectionModel().getSelectedItem();
        //affecter la valeur sélectionnée à la zone de texte
        texteArticle.setText(articleSelectionne);
        // donner le focus à la zone de saisie
        texteArticle.requestFocus();
    }
});
```



Utilisation d'un modèle ObservableList (1)

- Une deuxième solution pour utiliser un ListView consiste à utiliser le modèle MVC en stockant les données dans un objet de type **ObservableList<String>**

```
// créer un modèle de type ObservableList
ObservableList<String> modelArticles =FXCollections.observableArrayList();
//Initialiser le modèle par trois articles
modelArticles.addAll("Ordinateur","Clavier","SmartPhone");
//Instancier la vue "ListView" en lui passant le modèle
ListView<String> listeArticles = new ListView<String>(modelArticles);
```

- Dans le code du traitement de l'action du bouton, ajouter le nouvel article directement dans le modèle:

```
// gérer l'ajout d'un article saisi et son insertion dans la liste
boutonArticle.setOnAction( (e)->
{
    //récupérer le texte saisi par l'utilisateur dans le composant TextField
    String article= texteArticle.getText();
    //Insérer l'article dans le modèle
    modelArticles.add(article);
    // vider la zone de texte
    texteArticle.clear();
    // donner le focus à la zone de saisie
    texteArticle.requestFocus();
}
);
```

Utilisation d'un modèle ObservableList (2)

- En adoptant le motif MVC, utiliser cette fois un modèle de type `ObservableList<Article>` qui regroupe un ensemble d'objets de type «**Article**»:

```
public class Article {
    private int id;
    private String libelle;
    private double prix;
    private int qte;
    public Article() {}
    public Article(int id, String libelle, double prix, int qte)
    {
        this.id = id;
        this.libelle = libelle;
        this.prix = prix;
        this.qte = qte;
    }
    @Override
    public String toString() {
        return "Article [" + libelle + "]";
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
}
```

```
public String getLibelle() {
    return libelle;
}
public void setLibelle(String libelle) {
    this.libelle = libelle;
}
public double getPrix() {
    return prix;
}
public void setPrix(double prix) {
    this.prix = prix;
}
public int getQte() {
    return qte;
}
public void setQte(int qte) {
    this.qte = qte;
}
}
```

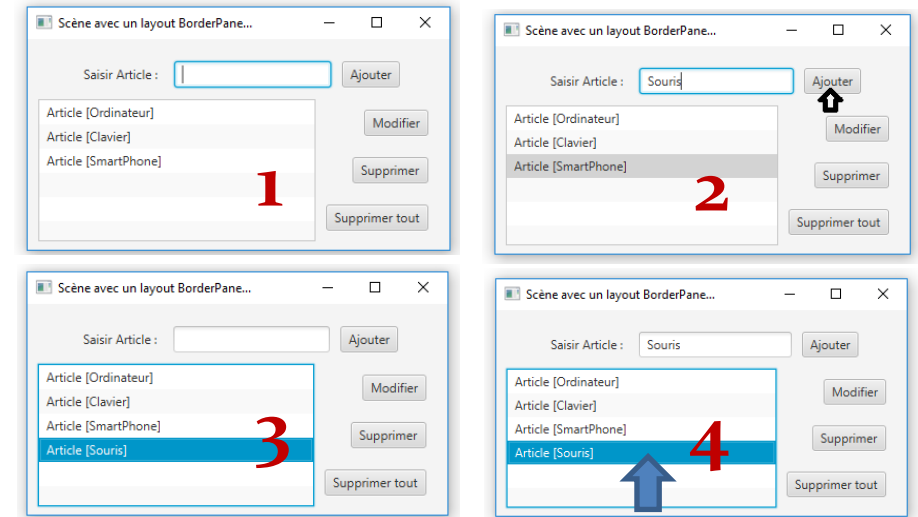
Utilisation d'un modèle ObservableList (3)

- En adoptant le motif MVC, utiliser cette fois un modèle de type `ObservableList<Article>` qui regroupe un ensemble d'objets de type « **Article** »:

```
// créer un modèle de type ObservableList<Article>
ObservableList<Article> modelArticles = FXCollections.observableArrayList();
//Initialiser le modèle par trois objets de type "Article"
Article a1 =new Article(1, "Ordinateur", 2000, 10);
Article a2 =new Article(2, "Clavier", 80, 20);
Article a3 =new Article(3, "SmartPhone", 3000, 5);
modelArticles.addAll(a1,a2,a3);
//Instancier la vue "ListView" en lui passant le modèle
ListView<Article> listeArticles = new ListView<Article>(modelArticles);
```

```
// gérer l'ajout d'un article saisi et son insertion dans la liste
boutonArticle.setOnAction( (e)->
{
    // récupérer le texte saisi par l'utilisateur
    // dans le composant TextField
    String libelle= texteArticle.getText();
    //créer une instance de type "Article"
    Article a = new Article();
    //Affecter la valeur du libellé
    a.setLibelle(libelle);
    //Insérer l'article dans le modèle
    modelArticles.add(a);
    // vider la zone de texte
    texteArticle.clear();
    // donner le focus à la zone de saisie
    texteArticle.requestFocus();
}
);
```

```
// Gérer la sélection d'un élément dans ListView
listeArticles.setOnMouseClicked( (e)->
{
    // s'il s'agit du bouton gauche de la souris et un double click
    if ( (e.getButton()==MouseButton.PRIMARY) && (e.getClickCount()==2))
    {
        //récupérer l'élément sélectionné
        Article articleSelectionne =
        listeArticles.getSelectionModel().getSelectedItem();
        //affecter la valeur sélectionnée à la zone de texte
        texteArticle.setText(articleSelectionne.getLibelle());
        // donner le focus à la zone de saisie
        texteArticle.requestFocus();
    }
}
);
```



Interfaces JavaFX basées sur FXML

- Il est possible, avec JavaFX, de définir la structure d'une application graphique à la base d'un fichier **XML**.
- Il suffit de déclarer l'arborescence des composants graphiques via des balises imbriquées (balises XML **personnalisées**)
- Le fichier XML qui décrit l'interface graphique porte l'extension **FXML** et représente ainsi la vue de l'application.
- L'application charge le fichier FXML, au démarrage, pour instancier les différents composants déclarés et ainsi l'interface graphique sera affichée.
- Pour gérer les événements, on associe chaque fichier FXML à une classe JAVA dite « **Contrôleur** » qui contient tous les traitements des événements.

Exemple d'une application JavaFX avec FXML

- Avec eclipse, installer un plugin nommé **e(fx)clipse**
- Créer un fichier FXML (**new FXML Document**) nommé **vue.fxml**
- Déclarer dans ce fichier la hiérarchie des composants graphiques (y compris les layouts) à partir du composant principal (dans ce cas **BorderLayout**)
- Associer le fichier FXML à une classe **Contrôleur** qui traite les événements
- A chaque type d'événement, associer au composant graphique le nom de la méthode correspondante définie dans la classe Contrôleur.
- Utiliser dans la classe Contrôleur une annotation **@FXML** pour lier les composants graphiques déclarés dans la classe Contrôleur et ceux déclarés dans le fichier vue.fxml

Fichier vue.fxml

```
<?xml version = "1.0" encoding = "UTF-8"?>
<?import javafx.scene.layout.BorderPane?>
<?import javafx.scene.layout.HBox?>
<?import javafx.scene.layout.VBox?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.ListView?>
<?import javafx.geometry.Insets?>
<?import javafx.collections.FXCollections?>
<?import java.lang.*?>

<BorderPane xmlns:fx="http://javafx.com/fxml/1"
fx:controller="exemple.Controleur">

    <top>
        <HBox spacing="10">
            <padding>
                <Insets left="10" top="10" right="10" bottom = "10">
                </Insets>
            </padding>
            <children>
                <Label text="Saisir un article"></Label>
                <TextField fx:id = "texteArticle"></TextField>
                <Button text="Ajouter"
                    onAction="#ajouterArticle"></Button>
            </children>
        </HBox>
    </top>
```

```
<center>
    <VBox spacing="10">
        <padding>
            <Insets left="10" top="10" right="10" bottom = "10">
            </Insets>
        </padding>
        <ListView fx:id = "listeArticles">
            <items>
                <FXCollections fx:factory="observableArrayList">
                    <String fx:value="Ordinateur"/>
                    <String fx:value="SmartPhone"/>
                    <String fx:value="Clavier"/>
                </FXCollections>
            </items>
        </ListView>
    </VBox>
</center>
</BorderPane>
```

Classe Controleur.java

```
package exemple;

import javafx.fxml.FXML;
import javafx.scene.control.ListView;
import javafx.scene.control.TextField;

public class Controleur {
    @FXML TextField texteArticle;
    @FXML ListView<String> listeArticles;

    public void ajouterArticle()
    {
        String article =texteArticle.getText();
        listeArticles.getItems().add(article);
    }
}
```

Classe MonApplication.java

```
package exemple;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.scene.layout.BorderPane;
import javafx.stage.Stage;

public class MonApplication extends Application{

    public static void main(String[] args) {
        Launch(args);
    }

    @Override
    public void start(Stage primaryStage) throws Exception {

        BorderPane root = FXMLLoader.Load(getClass().getResource("vue.fxml"));
        Scene scene= new Scene (root, 400,300);
        primaryStage.setScene(scene);
        primaryStage.setTitle("Java FX avec FXML....");
        primaryStage.show();
    }
}
```


Exercices

Exercice 1:

- Réaliser, dans un projet nommé `javaFX`, le code de l'exemple de gestion des articles (objets de type `Article`) vu dans ce cours avec une définition de l'interface graphique avec du code JAVA.
- Compléter la gestion des autres boutons:
 - Modifier
 - Supprimer
 - Supprimer Tout

Exercice2 :

- Réaliser la même application en décrivant l'interface graphique dans un fichier `vue.fxml` et en gérant les événements dans une classe `Contrôleur`